UNIT-III

CATALOGING AND INDEXING

Automatic Indexing: Classes of Automatic Indexing, Statistical Indexing, Natural Language, Concept Indexing, Hypertext Linkages
Document and Term Clustering: Introduction to Clustering, Thesaurus Generation, Item Clustering, Hierarchy of Clusters

Automatic Indexing

- The indexing process is a transformation of an item that extracts the semantics of the topics discussed in the item.
- The extracted information is used to create the processing tokens and the searchable data structure.
- The semantics of the item not only refers to the subjects discussed in the item but also in weighted systems.
- The index can be based on the full text of the item, automatic or manual generation of a subset of terms/phrases to represent the item, natural language representation of the item or abstraction to concepts in the item.
- The results of this process are stored in one of the data structures.

Classes of Automatic Indexing

- Automatic indexing is the process of analyzing an item to extract the information to be permanently kept in an index.
- This process is associated with the generation of the searchable data structures associated with an item.
- The left side of the figure including Identify Processing Tokens, Apply Stop Lists, Characterize tokens, Apply Stemming and Create Searchable Data Structure is all part of the indexing process.
- Filters, such as stop lists and stemming algorithms, are frequently applied to reduce the number of tokens to be processed.
- The next step depends upon the search strategy of a particular system.

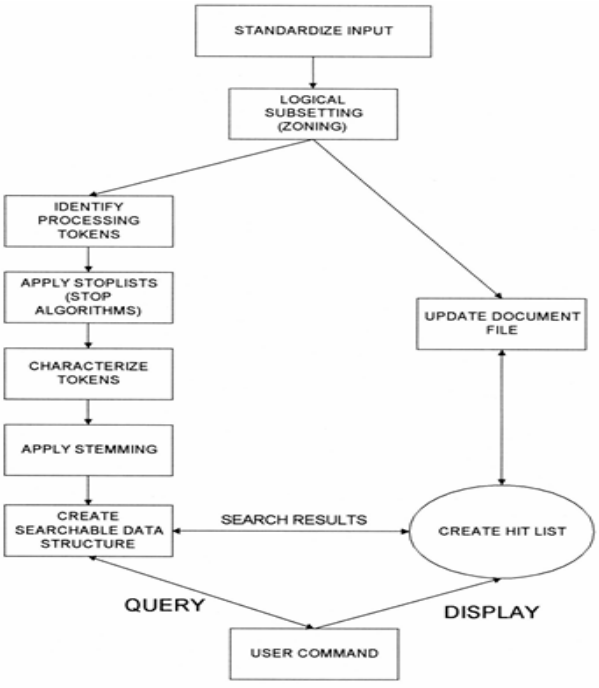
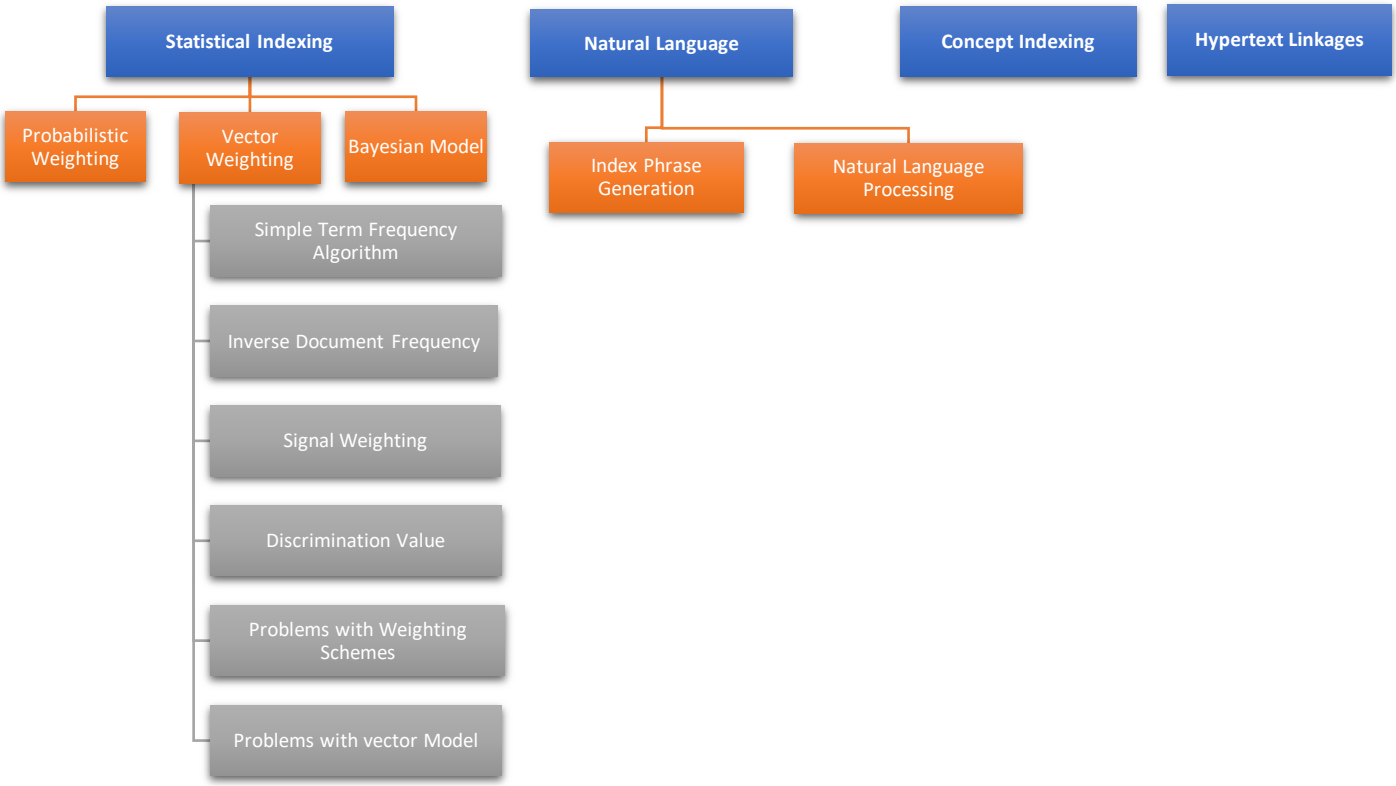


Figure 5.1 Data Flow in Information Processing System

- Search strategies can be classified as
  1. **Statistical,**
  2. **Natural Language,**
  3. **Concept.**
- An index is the data structure created to support the search strategy.
- **Statistical Indexing**
  - covers the broadest range of indexing techniques and common in commercial systems.
  - Basis for statistical is – frequency of occurrence of processing tokens (words/ phrases) within documents and within data base.
  - The words/phrases are the domain of searchable values.
  - Statistics that are applied to the event data are
    1. **Probabilistic,**
    2. **Bayesian,**
    3. **Vector spaces,**
    4. **Neural net.**
  - Statistic approach – stores a single statistic, such as how often each word occurs in an item – used for generating relevance scores after a Boolean search.



  - **Probabilistic indexing**
    - stores the information that are used in calculating a probability that a particular item satisfies (i.e., is relevant to) a particular query.

- **Bayesian and vector space**
  - stores the information that are used in generating a relative confidence level of an items relevancy to a query.
- **Neural networks**
  - dynamic learning structures– comes under concept indexing – that determine concept class.

- **Natural language approach**
  - perform the similar processing token identification as in statistical techniques - additional level of parsing of the item (present, past, future action) enhance search precision.
- **Concept indexing**
  - uses the words within an item to correlate to concepts discussed in the index item.
  - When generating the concept classes automatically, there may not be a name applicable to the concept but just a statistical significance.
  - Finally, a special class of indexing can be defined by creation of hypertext linkages.
  - These linkages provide virtual threads of concepts between items versus directly defining the concept within an item. Fig 5.1: Data flow in Information Processing System
  - Each technique has its own strengths and weaknesses.
- Uses frequency of occurrence of events to calculate the number to indicate potential relevance of an item.
- The documents are found by normal Boolean search and then statistical calculation are performed on the hit file, ranking the output(e.g. The term- frequency algorithm)
  1. **Probability weighting**
  2. **Vector Weighting**
     a. **Simple Term Frequency algorithm**
     b. **Inverse Document Frequency algorithm**
     c. **Signal Weighting**
     d. **Discrimination Value**
     e. **Problems with the weighting schemes and vector model**
  3. **Bayesian Model**

1. Probabilistic weighting:

- Probabilistic systems attempt to calculate a probability value that should be invariant to both calculation method and text corpora(large collection of written/spoken texts).
- The probabilistic approach is based on direct application of theory of probability to information retrieval system.
- **Advantage**: uses the probability theory to develop the algorithm.
  - This allows easy integration of the final results when searches are performed across multiple databases and use different search algorithms.
  - The use of probability theory is a natural choice – the basis of evidential reasoning (drawing conclusions from evidence).
  - This is summarized by PRP (probability ranking principle) and Plausible corollary (reasonable result)

- **HYPOTHESIS** –if a reference retrieval systems response to each request is a ranking of the documents in order of decreasing probability of usefulness to the user, the overall effectiveness of the system to the users is best obtainable on the basis of the data available.
- **PLAUSIBLE COROLLARY**: the techniques for estimating the probabilities of usefulness for output ranking in IR is standard probability theory and statistics.
- probabilities are based on binary condition the item is relevant or not.
- IRS the relevance of an item is a continuous function from non- relevant to absolutely useful.
- **Source of problems**: Probability theory come from lack of accurate data and simplified assumptions that are applied to mathematical modeling.
- cause the results of probabilistic approaches in ranking items to be less accurate than other approaches. Advantage of probabilistic approach is that it can identify its weak assumptions and work to strengthens them.
- Ex : **logistical regression**
- Approach starts by defining a model 0 system.
- In retrieval system there exits a query q_i and a document term di which has a set of attributes (Vi…Vn) from the query (e.g., counts of term frequency in the query), from the document (e.g., counts of term frequency in the document) and from the database (e.g., total number of documents in the database divided by the number of documents indexed by the term).
- The logistic reference model uses a random sample of query document terms for which binary relevance judgment has been made from judgment from samples.
- **Estimate Log-Odds :** Logarithm O is the logarithm of odds of relevance for terms T_k which is present in document D_j and query Q_i

$$\log(O(R \mid Q_i, D_j, t_k)) = c_0 + c_1 v_1 + \ldots + c_n v_n$$

This is for a single term t_k from query Q_i in document D_j.

$v_1, v_2, \ldots$: term frequency stats (features)

$c_0, c_1, \ldots$: learned regression coefficients

- **Aggregate Log-Odds Across All Query Terms:**

The logarithm that the **i**th Query is relevant to the **j**th Document is the sum of the logodds for all terms:

$$\log(O(R \mid Q_i, D_j)) = \sum_{k=1}^{q} [\log(O(R \mid Q_i, D_j, t_k)) - \log(O(R))]$$

q: Number of terms in query Q_i

$\log(O(R))$: Base log-odds of any document being relevant (a constant)

- **Convert Log-Odds to Probability**

  Using the **logistic function**, we convert log-odds to probability:

$$P(R \mid Q_i, D_j) = \frac{1}{1 + e^{-\log(O(R|Q_i,D_j))}}$$

  This gives us a value between 0 and 1—perfect for ranking!

  - The coefficients of the equation for logodds is derived for a particular database using a random sample of query-document-term-relevance quadruples and used to predict odds of relevance for other query-document pairs.
  - Additional attributes of relative frequency in the query (QRF), relative frequency in the document (DRF) and relative frequency of the term in all the documents (RFAD) were included, producing the logodds formula:
  - QRF = QAF\ (total number of terms in the query), DRF = DAF\(total number of words in the document) and RFAD = (total number of term occurrences in the database)\ (total number of all words in the database).

- **Feature-Based Model (Logistic Regression)**

Each term's contribution is calculated with specific features:

$$Z_j = \log(O(R \mid t_j)) = c_0 + c_1 \log(\text{QAF}) + c_2 \log(\text{QRF}) + c_3 \log(\text{DAF}) + c_4 \log(\text{DRF}) + c_5 \log(\text{IDF}) + c_6 \log(\text{RFAD})$$

Where:

- **QAF**: Query Absolute Frequency
- **QRF**: Query Relative Frequency
- **DAF**: Document Absolute Frequency
- **DRF**: Document Relative Frequency
- **IDF**: Inverse Document Frequency
- **RFAD**: Relative Frequency Across Documents

Feature	Symbol	Value	Formula	Meaning
Query Absolute Frequency	QAF	2	Count of term t_1 in query Q_1	Number of times "intelligence" appears in the query
Query Relative Frequency	QRF	0.4	QAF / Total number of terms in query	Proportion of "intelligence" in the query
Document Absolute Frequency	DAF	4	Count of term t_1 in document D_1	Number of times "intelligence" appears in Document D_1
Document Relative Frequency	DRF	0.02	DAF / Total number of terms in document	Proportion of "intelligence" in Document D_1
Inverse Document Frequency	IDF	1.8	log(Total Documents / Documents containing t_1)	Measures rarity of term in the corpus
Relative Frequency Across Docs	RFAD	0.005	Avg(DRF across all documents containing t_1)	Average DRF of "intelligence" across documents where it appears

- **Final Ranking Formula**
  - Logs are used to reduce the impact of frequency information; then smooth out skewed distributions.
  - A higher max likelihood is attained for logged attributes.
  - The coefficients and log (O(R)) were calculated creating the final formula for ranking for query vector Q', which contains q terms:

The final formula to rank a document for a given query is:

$$\log(O(R \mid \tilde{Q})) = -5.138 + \sum_{k=1}^{q} (Z_j + 5.138)$$

This adds all the contributions Z_j from the query terms, adjusted by a base constant -5.138.

  - The logistic inference method was applied to the test database along with the Cornell SMART vector system, inverse document frequency and cosine relevance weighting formulas.
  - The logistic inference method outperformed the vector method.
  - Attempts have been made to combine different probabilistic techniques to get a more accurate value.
  - This combination of logarithmic odds has not presented better results.
  - The objective is to have the strong points of different techniques compensate for weaknesses.
  - To date this combination of probabilities using averages of Log-Odds has not produced better results and in many cases produced worse results

- **Example:**

We want to score the relevance of **Document** `D1` to `Q1`.

Let's say we extracted a **term** `t1` = "**intelligence**" with the following values:

Feature	Symbol	Value
Query Absolute Freq	QAF	2
Query Relative Freq	QRF	0.4
Document Abs Freq	DAF	4
Document Relative Freq	DRF	0.02
Inverse Doc Freq	IDF	1.8
Rel Freq Across Docs	RFAD	0.005

Assume the logistic regression model was trained and returned these coefficients:

Coefficient	Value
c_0	-0.5
c_1	0.8
c_2	1.2
c_3	0.6
c_4	0.5
c_5	1.1
c_6	-0.7

✳ Step 1: Compute `Z_j = log(O(R | t_j))`

Using:

$$Z_j = c_0 + c_1 \log(QAF) + c_2 \log(QRF) + c_3 \log(DAF) + c_4 \log(DRF) + c_5 \log(IDF) + c_6 \log(RFAD)$$

Now compute each log:

```
text

log(2) ≈ 0.301
log(0.4) ≈ -0.398
log(4) ≈ 0.602
log(0.02) ≈ -1.699
log(1.8) ≈ 0.588
log(0.005) ≈ -2.301
```

$$Z_j = -0.5 + 0.8(0.301) + 1.2(-0.398) + 0.6(0.602) + 0.5(-1.699) + 1.1(0.588) + (-0.7)(-2.301)$$

$$Z_j \approx -0.5 + 0.241 - 0.478 + 0.361 - 0.849 + 0.647 + 1.611$$

$$Z_j \approx 1.033$$

Step 2: Convert Log-Odds to Probability

Use the logistic (sigmoid) function:

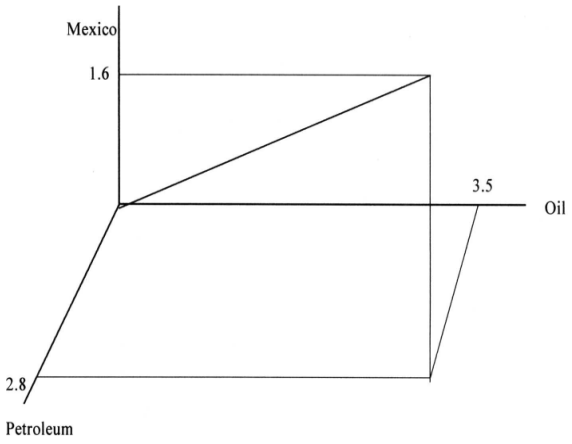
$$P(R \mid Q, D) = \frac{1}{1 + e^{-Z_j}} = \frac{1}{1 + e^{-1.033}} \approx 0.737$$

2. Vector Weighting

- Earliest system that investigated statistical approach is SMART system of Cornell university. – system based on vector model.
- Vector is one dimension of set of values, where the order position is fixed and represents a domain.
- Each position in the vector represents a processing token.
- Two approaches to the domain values in the vector – binary or weighted
- Under binary approach the domain contains a value of 1 or 0
- Under weighted - domain is set of positive value – the value of each processing token represents the relative importance of the item.
- Binary vector requires a decision process to determine if the degree that a particular token the semantics of item is sufficient to include in the vector.
- Ex., a five-page item may have had only one sentence like “Standard taxation of the shipment of the oil to refineries is enforced.”
- For the binary vector, the concepts of “Tax” and “Shipment” are below the threshold of importance (e.g., assume threshold is 1.0) and they not are included in the vector.

	Petroleum	Mexico	Oil	Taxes	Refineries	Shipping
Binary	(1 ,	1 ,	1 ,	0 ,	1 ,	0)
Weighted	(2.8 ,	1.6 ,	3.5 ,	.3 ,	3.1 ,	.1)

- A Weighted vector acts same as the binary vector but provides a range of values that accommodates a variance in the value of relative importance of processing tokens in representing the item.
- The use of weights also provides a basis for determining the rank of an item.
- The vector approach allows for mathematical and physical representation using a vector space model.
- Each processing token can be considered another dimension in an item representation space.
- 3D vector representation assuming there were only three processing tokens, Petroleum Mexico and Oil.
    - This figure shows a **3D coordinate space** with just 3 dimensions:
        - X-axis: Petroleum
        - Y-axis: Mexico
        - Z-axis: Oil
    - The **document vector** is a point in this space:
      (2.8, 1.6, 3.5)
    - Each document is represented as such a point or arrow in **an n-dimensional space.**



2. Vector Weighting

a) Simple Term Frequency Algorithm (STFA)

- In both weighted and un weighted approaches an automatic index processing implements an algorithm to determine the weight to be assigned to a processing token.
- **In statistical system**: the data that are potentially available for calculating a weight are the frequency of occurrence of the processing token in an existing item (i.e., term frequency - TF), the frequency of occurrence of the processing token in the existing database (i.e., total frequency -TOTF) and the number of unique items in the database that contain the processing token (i.e., item frequency - IF, frequently labeled in other document frequency - DF).
- Simplest approach is to have the weight equal to the term frequency.
- If the word “computer” occurs 15 times within an item it has a weight of 15.
- The term frequency weighting formula: (1+log(TF))/1+log(average(TF)) / (1-slope) * pivot + slope*no. of unique terms
- where slope was set at .2 and the pivot was set to the average no of unique terms occurring in the collection.
- Slope and pivot are constants for any document/query set.
- This leads to the final algorithm that weights each term by the above formula divided by the pivoted normalization:

$$\frac{(1 + \log(TF))/1 + \log(\text{average } (TF))}{(1 - \text{slope}) * \text{pivot} + \text{slope} * \text{number of unique terms}}$$

STFA Example

TF (Term Frequency) = 5

Average TF = 3

Slope = 0.7

Number of unique terms = 100

Pivot = 80

Step 1: Calculate the numerator

$$1 + \log_{10}(TF) = 1 + \log_{10}(5) \approx 1.699$$

Step 2: Calculate the denominator

- $\log_{10}(\text{Average TF}) = \log_{10}(3) \approx 0.477$
- So,

$$\frac{1 + \log_{10}(3)}{0.7 \times 100} + (1 - 0.7) \times 80 = \frac{1.477}{70} + 0.3 \times 80 = 0.0211 + 24 = 24.0211$$

Step 3: Final Normalized TF

$$\text{Normalized TF} = \frac{1.699}{24.0211} \approx 0.0707$$

2. Vector Weighting

b) Inverse Document Frequency Algorithm (IDF)

- IDF measures how unique or rare a term is across all documents in a corpus. It is part of the **TF-IDF** (Term Frequency-Inverse Document Frequency) weighting scheme, which is widely used to rank documents based on their relevance to a query.
- Enhancing the weighting algorithm: the weights assigned to the term should be inversely proportional to the frequency of term occurring in the data base.
- The term “computer” represents a concept used in an item, but it does not help a user find the specific information being sought since it returns the complete DB.
- This leads to the general statement enhancing weighting algorithms that the weight assigned to an item should be inversely proportional to the frequency of occurrence of an item in the database.
- This algorithm is called inverse document frequency (IDF).
- The un-normalized weighting formula is:

$$\text{WEIGHT}_{ij} = \text{TF}_{ij} * [\text{Log}_2(n) - \text{Log}_2(\text{IF}_j) + 1]$$

Where:
TF$_{ij}$: Term frequency of term *j* in document *i*
n: Total number of items/documents
IF$_j$: Number of items/documents containing term *j*

- **IDF Example**

- Total items (n) = 2048
- Terms & Frequencies:

- "oil": appears 4 times in the item, found in 128 documents
- "Mexico": appears 8 times, found in 16 documents
- "refinery": appears 10 times, found in 1024 documents

1. Weight for "oil":

$$TF = 4, \quad IF_{oil} = 128 \Rightarrow \log_2(128) = 7$$

$$\text{Weight}_{oil} = 4 \times (11 - 7 + 1) = 4 \times 5 = \boxed{20}$$

2. Weight for "Mexico":

$$TF = 8, \quad IF_{Mexico} = 16 \Rightarrow \log_2(16) = 4$$

$$\text{Weight}_{Mexico} = 8 \times (11 - 4 + 1) = 8 \times 8 = \boxed{64}$$

3. Weight for "refinery":

$$TF = 10, \quad IF_{refinery} = 1024 \Rightarrow \log_2(1024) = 10$$

$$\text{Weight}_{refinery} = 10 \times (11 - 10 + 1) = 10 \times 2 = \boxed{20}$$

$\text{Weight}_{oil} = 4 * (\text{Log}_2(2048) - \text{Log}_2(128) + 1) = 4 * (11 - 7 + 1) = 20$

$\text{Weight}_{Mexico} = 8 * (\text{Log}_2(2048) - \text{Log}_2(16) + 1) = 8 * (11 - 4 + 1) = 64$

$\text{Weight}_{refinery} = 10 * (\text{Log}_2(2048) - \text{Log}_2(1024) + 1) = 10 * (11 - 10 + 1) = 20$

with the resultant inverse document frequency item vector = (20, 64, 20)

2. Vector Weighting

c) Signal weighting

- Signal Weighting Can be used to improve **term weighting** in **information retrieval systems (IRS)** beyond standard inverse document frequency (IDF).
- The **signal weighting method** enhances **precision** in search results by considering **how a term is distributed across documents**, not just how many documents contain it. This adds a **statistical depth** using shannon's information theory to standard retrieval models.
- **Information theory**—specifically **shannon's entropy**
- IDF adjusts the weight of a processing token for an item based upon the number of items that contain the term in the existing database.
- It does not account for is the term frequency distribution of the processing token in the items that contain the term - can affect the ability to rank items.
- For example, assume the terms “SAW” and “DRILL” are found in 5 items with the following frequencies:

Item Distribution	SAW	DRILL
A	10	2
B	10	2
C	10	18
D	10	10
E	10	18

Figure 5.5 Item Distribution for SAW and DRILL

- In Information Theory, the information content value of an object is inversely proportional to the probability of occurrence of the item.
- An instance of an event that occurs all the time has less information value than an instance of a seldom occurring event.

Weight Calculation:

1. **Information Value**:
   - Defined as:

$$\text{INFORMATION} = -\log_2(p)$$

   where p is the probability of an event (e.g., a term occurring in a document).
   - Rare events (lower p) have **higher information value**.
2. **Average Information (Entropy)**:
   - If a term appears with **equal frequency** across all documents, its average information content is **maximized**:

$$\text{AVE\_INFO} = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

3. **Signal Weight**:
   - Designed to capture **distribution variance**:

$$\text{Signal}_k = \log_2(\text{TOTF}) - \text{AVE\_INFO}$$

   - Where TOTF is the total frequency of a term across all documents.
   - Higher **Signal** = more **concentrated or skewed** distribution = more **informative**.

Weight Calculation for SAW and DRILL:

Term: SAW

Distribution in items A to E: [10, 10, 10, 10, 10]

Total Frequency (TOTF) = 50

- Each item has the same frequency → **uniform distribution**
- So:

$$\text{AVE\_INFO} = -\sum\left(\frac{10}{50}\log_2\frac{10}{50}\right) = -5\left(\frac{1}{5}\log_2\frac{1}{5}\right) = \log_2(5)$$

$$\text{Signal}_{SAW} = \log_2(50) - \log_2(5) = \log_2(10)$$

Term: DRILL

Distribution: [2, 2, 2, 10, 18, 10, 6] (hypothetical interpretation of image data)

Total Frequency = 50

- Distribution is **non-uniform** → higher Signal
- The calculated AVE_INFO is lower → Signal is higher

2. Vector Weighting

d) Discrimination Value Weighting

- The goal of this weighting approach is to assign importance to terms that help **distinguish (discriminate)** between documents/items in a collection.
- The **Discrimination Value (DISCRIM)** measures how much a term contributes to differentiating items.
- Creating a weighting algorithm based on the discrimination of value of the term but all items appear the same, the harder it is to identify those that are needed.

- Salton and Yang proposed a weighting algorithm that takes into consideration the ability for a search term to discriminate among items.

Steps to calculate weight using discrimination value:

- **Step 1**: Compute $DISCRIM_i$

$$DISCRIM_i = AVESIM_i - AVESIM$$

- AVESIM = average similarity between all items in the database.
- $AVESIM_i$ = average similarity when term i is removed from all items.
  - $DISCRIM_i$ value being positive, close to zero/negative.
  - A positive value indicates that removal of term "i" has increased the similarity between items.
  - In this case, leaving the term in the database assists indiscriminating between items and is of value.
  - A value close to zero implies that the term's removal or inclusion does not change the similarity between items.
  - If the value is negative, the term's effect on the database is to make the items appear more similar since their average similarity decreased with its removal.

- **Step 2**: Compute Final Term Weight

$$Weight_{ik} = TF_{ik} * DISCRIM_k$$

- TF_{i} = Term frequency of term k in document i.
- $DISCRIM$ = Discrimination value for term k.

Example

Term	Doc1 TF	Doc2 TF	Doc3 TF	AVESIM (no term)	$DISCRIM_i$ = $AVESIM_i$ – 0.559	Normalized $DISCRIM_i$	Doc1 Weight	Doc2 Weight	Doc3 Weight
T1	2	1	0	0.714	0.155	0.155	2 × 0.155 = 0.310	1 × 0.155 = 0.155	0 × 0.155 = 0
T2	1	1	2	0.447	–0.112	0	1 × 0 = 0	1 × 0 = 0	2 × 0 = 0
T3	0	2	1	0.701	0.142	0.142	0 × 0.142 = 0	2 × 0.142 = 0.284	1 × 0.142 = 0.142

Calculating Cosine Similarity with Example

Cosine similarity measures the **angle** between two vectors. In the context of documents, the closer the angle (i.e., smaller), the more similar the documents.

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\|\|B\|}$$

- $A \cdot B$ = dot product of vectors A and B
- $\|A\|$ = magnitude (length) of vector A
- Value ranges from **0** (**completely dissimilar**) to **1** (**identical**)

Vectors:

- Doc1: $A = [2, 1, 0]$
- Doc2: $B = [1, 1, 2]$

Dot Product:

$$A \cdot B = (2 \times 1) + (1 \times 1) + (0 \times 2) = 2 + 1 + 0 = 3$$

Magnitudes:

- $\|A\| = \sqrt{2^2 + 1^2 + 0^2} = \sqrt{5} \approx 2.236$
- $\|B\| = \sqrt{1^2 + 1^2 + 2^2} = \sqrt{6} \approx 2.449$

Cosine Similarity:

$$\frac{3}{2.236 \times 2.449} \approx \frac{3}{5.477} \approx 0.547$$

2. Vector Weighting

e) Problems With Weighting Schemes

- Often weighting schemes use information that is based upon processing token distributions across the database.
- Information databases tend to be dynamic with new items always being added and to a lesser degree old items being changed or deleted.
- Thus, these factors are changing dynamically.
- This presents a challenge in maintaining consistent and accurate weightings.

Three Main Approaches to Handle Changing Weight Factors:

Aspect	**Approach 1**: Rebuild Periodically	**Approach 2**: Threshold-Based Update	**Approach 3**: Dynamic Calculation
Method	Use current values; recalculate entire database periodically	Use fixed values; update only when change threshold is exceeded	Store invariant values; compute weights during search
Overhead	Low during runtime; high during rebuild	Moderate; distributed over time	High computation per query (minimal if using inverted files)
Accuracy	Medium – fluctuates over time	High – stable until major changes	Very High – always current
Best For	Simpler systems, infrequent updates	Balanced efficiency and accuracy	High-accuracy systems or small databases

Downside	Costly rebuilds for large DBs	Complexity in tracking changes	Slower search times without optimization

- SOLUTIONS:
  - If the system is using an inverted file search structure, this overhead is very minor.
  - The best environment would allow a user to run a query against multiple different time periods and different databases that potentially use different weighting algorithms, and have the system integrate the results into a single ranked Hit file.

Problems With the Vector Model

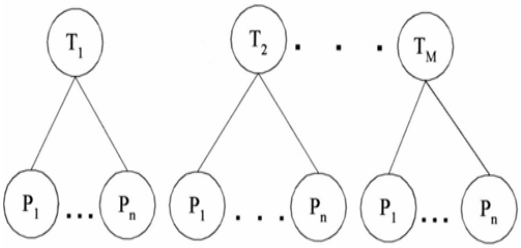
Issue Category	Problem Description	Example / Explanation
Dynamic Databases	Weighting factors can become inaccurate as the database content changes dynamically.	Term importance may shift over time, but the model doesn't adapt automatically.
Multiple Topics in a Document	The model cannot differentiate between distinct topics discussed in the same document.	A document discussing “oil in Mexico” and “coal in Pennsylvania” may match “coal in Mexico” incorrectly.
Lack of Term Association	Terms are treated independently; no correlation or linkage between related terms (no precoordination).	“Coal” and “Mexico” are scored independently, even if unrelated in context.
No Positional Information	Cannot perform proximity searches (e.g., term A within 10 words of term B).	Positional relationships between terms are not stored.
Scalar Value Limitation	Each term is assigned only one scalar value per document — no detail about term location or section.	Can't distinguish where the term appears in the document.
Subset Searching as a Fix	Searching within parts (subsets) of a document can improve precision by focusing on specific topics.	Helps isolate sections that match a search query more accurately.

3. **Bayesian Model**

- One way of overcoming the restrictions inherent in a vector model is to use a Bayesian approach to maintaining information on processing tokens.
- The Bayesian model provides a conceptually simple yet complete model for information systems.
- The Bayesian approach is based upon conditional probabilities (e.g., Probability of Event 1 given Event 2 occurred).
- This general concept can be applied to the search function as well as to creating the index to the database.
- The objective of information systems is to return relevant items.
- The formula used is:

$$P(\mathrm{REL}/\mathrm{DOC}_i, \mathrm{Query}_j)$$

- The probability that a document (doc_i) is relevant (REL) given a query ($query_j$).
- In addition to search, Bayesian formulas can be used in determining the weights associated with a particular processing token in an item.
- The objective of creating the index to an item is to represent the semantic information in the item.
- A Bayesian network can be used to determine the final set of processing tokens (called topics) and their weights.



- A simple view of the process where Ti represents the relevance of topic "i" in a particular item and Pj represents a statistic associated with the event of processing token "j" being present in the item.
- "m" topics would be stored as the final index to the item.

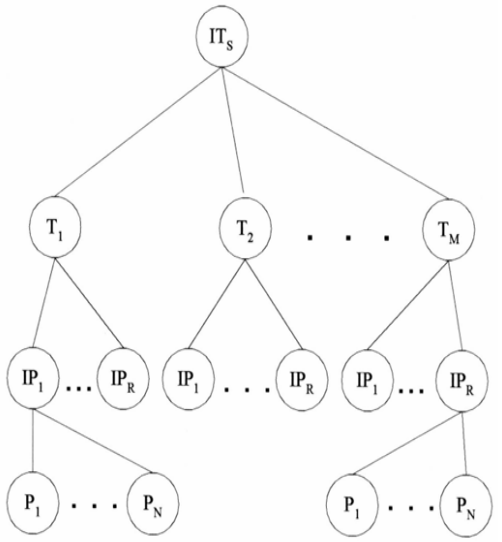
- The statistics associated with the processing token are typically frequency of occurrence.
- But they can also incorporate proximity factors that are useful in items that discuss multiple topics.
- There is one major assumption made in this model:

Assumption of Binary Independence:

- The topics and the processing token statistics are independent of each other.
- The existence of one topic is not related to the existence of the other topics.
- The existence of one processing token is not related to the existence of other processing tokens
- In most cases this assumption is not true. Some topics are related to other topics and some processing tokens related to other processing tokens.
- For example, the topics of "Politics" and "Economics" are in some instances related to each other and in many other instances totally unrelated.
- The same type of example would apply to processing tokens.

There are two approaches to handling this Binary Independence problem.

- The first is to assume that there are dependencies, but that the errors introduced by assuming the mutual independence do not noticeably effect the determination of relevance of an item nor its relative rank associated with other retrieved items.This is the most common approach used in system implementations.
- A second approach can extend the network to additional layers to handle interdependencies. Thus, an additional layer of Independent Topics (ITs) can be placed above the Topic layer and a layer of Independent Processing Tokens (IPs) can be placed above the processing token layer.



- Top level (ITs) could represent the item or system being indexed.
- The next level (T_1 to T_M) might be topics or thematic categories.
- Below that, IP_1 to IP_R might represent index phrases derived from processing.
- At the bottom, P_1 to P_N could be individual phrases or words.
- It shows how complex semantic processing builds up from base phrases to structured thematic indexes.

- The new set of Independent Processing Tokens can then be used to define the attributes associated with the set of topics selected to represent the semantics of an item.
- To compensate for dependencies between topics the final layer of Independent Topics is created.
- The degree to which each layer is created depends upon the error that could be introduced by allowing for dependencies between Topics or Processing Tokens.
- Although this approach is the most mathematically correct, it suffers from losing a level of precision by reducing the number of concepts available to define the semantics of an item.

Example

```
ITs (Item or Information to be searched)
       /     |     \
     T1     T2     TM   ← Different thematic categories (like "Desserts", "Healthy Recipes")
    /|\    /|\    /|\
 IP1...IPR IP1...IPR...  ← Index phrases like "sugar-free", "apple pie", "gluten-free"
   /         \
P1...PN     (words, phrases)
```

Natural language:

- The goal of natural language processing is to use the semantic information in addition to the statistical information to enhance the indexing of the item.
- This improves the precision of searches, reducing the number of false hits a user reviews.
- The semantic information is extracted as a result of processing the language rather than treating each word as an independent entity.
- The simplest output of this process results in generation of phrases that become indexes to an item.
- More complex analysis generates thematic representation of events rather than phrases.
- Statistical approaches use proximity as the basis behind determining the strength of word relationships in generating phrases.
- For example, with a proximity constraint of adjacency, the phrases “venetian blind” and “blind Venetian” may appear related and map to the same phrase.
- But syntactically and semantically those phrases are very different concepts.
- Word phrases generated by natural language processing algorithms enhance indexing specification and provide another level of disambiguation.
- Natural language processing can also combine the concepts into higher level concepts sometimes referred to as thematic representations.

Step No.	Step Name	Purpose
1	Lexical Analysis	Preprocessing: word forms, tense, plurality
2	Generation of Term Phrases	Identify basic important terms
3	Mapping to Subject Codes (LDOCE)	Assign semantic categories to terms
4	Text Structuring	Organize text into logical sections (Evaluation, Main Event, Expectation, etc.)
5	Topic Statement Identification	Find main ideas and their semantic attributes (time frame: past, present, future)
6	Assign News Schema Components	Assign sentences to categories like Circumstance, Consequence, Lead
7	Concept Relationship Detection	Identify how different topics/concepts relate
8	Relationship Weighting	Assign importance to relationships (e.g., active verbs weighted higher)
9	Storage for Retrieval	Store all information to help in natural language search queries

1. Index Phrase Generation

- The goal of indexing is to represent the semantic concepts of an item in the information system to support finding relevant information.
- Single words have conceptual context, but frequently they are too general to help the user find the desired information.
- Term phrases allow additional specification and focusing of the concept to provide better precision and reduce the user's overhead of retrieving non-relevant items.
- Having the modifier "grass" or "magnetic" associated with the term "field" clearly disambiguates between very different concepts.
- One of the earliest statistical approaches to determining term phrases using of a COHESION factor between terms :

$$COHESION_{k,h} = SIZE\text{-}FACTOR * (PAIR\text{-}FREQ_{k,h} / TOTF_k * TOTF_H)$$

- where **SIZE-FACTOR** is a normalization factor based upon the size of the vocabulary and is the total frequency of co-occurrence of the pair $Term_k$, $Term_h$ in the item collection.
- Co-occurrence may be defined in terms of adjacency, word proximity, sentence proximity, etc.
- This initial algorithm has been modified in the SMART system to be based on the following guidelines
    - Any pair of adjacent non-stop words is a potential phrase
    - Any pair must exist in 25 or more items
    - Phrase weighting uses a modified version of the smart system single term algorithm
    - Normalization is achieved by dividing by the length of the single-term sub-vector.
- Statistical approaches tend to focus on two term phrases.
- The natural language approaches are their ability to produce multiple-term phrases to denote a single concept.
- If a phrase such as "industrious intelligent students" was used often, a statistical approach would create phrases such as "industrious intelligent" and "intelligent student."
- A natural language approach would create phrases such as "industrious student," "intelligent student" and "industrious intelligent student."
- The first step in a natural language determination of phrases is a lexical analysis of the input.
- In its simplest form this is a part of speech tagger that, for example, identifies noun phrases by recognizing adjectives and nouns.
- Precise part of speech taggers exist that are accurate to the 99 per cent range.
- Additionally, proper noun identification tools exist that allow for accurate identification of names, locations and organizations since these values should be indexed as phrases and not undergo stemming.
- The Tagged Text Parser (TTP), based upon the Linguistic String Grammar, produces a regularized parse tree representation of each sentence reflecting the predicate-argument structure.
- The tagged text parser contains over 400 grammar production rules. Some examples of

CLASS	EXAMPLES
determiners	a, the
singular nouns	paper, notation, structure, language
plural nouns	operations, data, processes
preposition	in, by, of, for
adjective	high, concurrent
present tense verb	presents, associates
present participal	multiprogramming

- The TTP parse trees are header-modifier pairs where the header is the main concept and the modifiers are the additional descriptors that form the concept and eliminate ambiguities.
  - Example: The former Soviet President has been a local hero ever since a Russian tank invaded Wisconsin

```
|assert
  perf[HAVE]
    verb[BE]
      subject
        np
          noun[President]
          t_pos[The]
          adj[former]
          adj[Soviet]
      object
        np
          noun[hero]
          t_pos[a]
          adj[local]
      adv[ever]
      sub_ord
        [since]
          verb[invade]
            subject
              np
                noun [tank]
                t_pos[a]
                adj[Russian]
            object
              np
                noun[Wisconsin]
```

- To determine if a header-modifier pair warrants indexing, Strzalkowski calculates a value for Informational Contribution (IC) for each element in the pair.
  - The basis behind the IC formula is a conditional probability between the terms. The formula for IC between two terms (x,y) is:

$$IC(x,[x,y]) = \frac{fx,y}{Nx + Dx - 1}$$

IC(x, [x,y]): This is the *Index Contribution* of word `x` in the phrase `[x, y]`. It tells us how much word `x` contributes when used in that two-word phrase.

$f_{x,y}$: Frequency of the phrase `[x, y]` (how many times `[x, y]` appears).

N_x: Total number of bigrams (two-word combinations) that start with `x`.

D_x: Number of distinct second words that come after `x` in any phrase.

Example:

Let's take the bigram (two-word phrase):
`[data, mining]`

Assume:

- $f_{x,y}$ = f[data, mining] = 50 (The phrase "data mining" appears **50 times**)
- $N_x = 200$ (There are **200** bigrams starting with "data", like "data analysis", "data science", etc.)
- $D_x = 25$ (There are **25 unique** second words that come after "data")

Now plug into the formula:

$$IC(data, [data, mining]) = \frac{50}{200 + 25 - 1} = \frac{50}{224} \approx 0.223$$

Calculating Weight of Phrase

$$\text{weight}(\text{Phrase}_i) = (C_1 * \log(\text{termf}) + C_2 * \alpha(N,i)) * \text{IDF}$$

weight(Phrase$_i$): The overall weight or importance of a phrase `i`.

termf: Term frequency — how often the phrase appears.

log(termf): Logarithm of the frequency — to scale the value down but keep the differences.

α(N, i): A function that adjusts based on the position or length of the phrase (depends on implementation).

IDF: Inverse Document Frequency — it reduces the weight for very common phrases and boosts rare but informative ones.

C_1, C_2: Constants to balance the formula — how much weight you give to frequency vs structure.

2. Natural Language Processing:

- Natural language processing not only produces more accurate term phrases, but can provide higher level semantic information identifying relationships between concepts.
- System adds the functional processes Relationship Concept Detectors, Conceptual Graph Generators and Conceptual Graph Matchers that generate higher level linguistic relationships including semantic and is course level relationships.
- During the first phase of this approach, the processing tokens in the document are mapped to Subject Codes.
- These codes equate to index term assignment and have some similarities to the concept-based systems.
- The next phase is called the Text Structure, which attempts to identify general discourse level areas within an item.
- The next level of semantic processing is the assignment of terms to components, classifying the intent of the terms in the text and identifying the topical statements.
- The next level of natural language processing identifies interrelationships between the concepts.

- The final step is to assign final weights to the established relationships.
- The weights are based upon a combination of statistical information and values assigned to the actual words used in establishing the linkages.

Concept indexing

- Natural language processing starts with a basis of the terms within an item and extends the information kept on an item to phrases and higher-level concepts such as the relationships between concepts.
- Concept indexing takes the abstraction a level further.
- Its goal is using concepts instead of terms as the basis for the index, producing a reduced dimension vector space.
- Concept indexing can start with a number of unlabeled concept classes and let the information in the items define the concepts classes created.
- A term such as “automobile” could be associated with concepts such as “vehicle,” “transportation,” “mechanical device,” “fuel,” and “environment.”
- The term “automobile” is strongly related to “vehicle,” lesser to “transportation” and much lesser the other terms.
- Thus, a term in an item needs to be represented by many concept codes with different weights for a particular item.
- The basis behind the generation of the concept approach is a neural network model.
- Special rules must be applied to create a new concept class.
- Example demonstrates how the process would work for the term “automobile.”

TERM: automobile

	Weights for associated concepts:
Vehicle	.65
Transportation	.60
Environment	.35
Fuel	.33
Mechanical Device	.15

Vector Representation Automobile: (.65,..., .60, ..., .35, .33, ... , .15)

- Another example of this process is Latent Semantic Indexing (LSI).
- Its assumption is that there is an underlying or “latent” structure represented by interrelationships between words.
- The index contains representations of the “latent semantics” of the item. Like Convectis, the large term-document matrix is decomposed into a small set (e.g., 100-300) of orthogonal factors which use linear combinations of the factors (concepts) to approximate the original matrix.
- Any rectangular matrix can be decomposed into the product of three matrices. Let X be a mxn matrix such that:

$$X = T_0 \bullet S_0 \bullet D_0'$$

- Where T_0 and D_0 have orthogonal columns and are m x r and r x n matrices, S_0 is an r x r diagonal matrix and r is the rank of matrix X.

Hypertext linkages

- It's a new class of information representation is evolving on the Internet.
- Need to be generated manually Creating an additional information retrieval dimension.
- Traditionally the document was viewed as two dimensional.
- Text of the item as one dimension and references as second dimension.
- Hypertext with its linkages to additional electronic items, can be viewed as networking between the items that extend contents, i.e by embedding linkage allows the user to go immediately to the linked item.
- Issue: how to use this additional dimension to locate relevant information.
- At the internet we have three classes of mechanism to help find information.

1. Manually generated indexes ex: www.yahoo.com were information sources on the home page are indexed manually into hyperlink hierarchy.
    - The user navigates through hierarchy by expanding the hyper link.
    - At some point the user starts to see the end of items.
2. Automatically generated indexes - sites like lycos.com and altavista.com automatically go to other internet sites and return the text , ggogle.com.
3. Web Crawler's : A web crawler (also known as a Web spider or Web robot) is a program or automated script which browses the World Wide Web in a methodical, automated manner.
    - Webcrawlers (e.g., WebCrawler, OpenText, Pathfinder) and intelligent agents (Coriolis Groups' NetSeeker™) are tools that allow a user to define items of interest and they automatically go to various sites on the Internet searching for the desired information.

- What is needed is an index algorithm for items that looks at the hypertext linkages as an extension of the concepts being presented in the item where the link exists.
- Some links that are for references to multi-media imbedded objects would not be part of the indexing process.
- The Universal Reference Locator (URL) hypertext links can map to another item or to a specific location within an item.
- The index values of the hyperlinked item have a reduced weighted value from contiguous text biased by the type of linkage. The weight of processing tokens appears:

$$\mathbf{Weight_{i,j,k,l}} = (\alpha * \mathbf{Weight_{i,j}} + \beta * \mathbf{Weight_{k,l}}) * (\gamma * \mathbf{Link_{i,k}})$$

$\mathrm{Weight}_{i,j}$: Importance or frequency of token j in **item i** (the current document).

$\mathrm{Weight}_{k,l}$: Importance or frequency of token l in **item k** (the hyperlinked document).

$\mathrm{Link}_{i,k}$: The strength of the hyperlink from **item i to item k**.

α, β: Normalization or scaling factors that adjust the relative importance of the current document's and linked document's tokens.

γ: A weighting factor that adjusts for the **strength or relevance of the hyperlink**.

Introduction to Clustering

- Clustering is the process of **grouping similar items or terms** together to improve how we search and retrieve information.
- The goal of the clustering was to assist in the location of information.
- Clustering of words originated with the generation of thesauri. Thesaurus, coming from the Latin word meaning "treasure," is similar to a dictionary in that it stores words.
- Instead of definitions, it provides the synonyms and antonyms for the words.
- Its primary purpose is to assist authors in selection of vocabulary.
- The goal of clustering is to provide a grouping of similar objects (e.g., terms or items) into a "class" under a more general title. Clustering also allows linkages between clusters to be specified.
- The term class is frequently used as a synonym for the term cluster.

Clustering Process Steps:

- Define the domain for the clustering effort. Defining the domain for the clustering identifies those objects to be used in the clustering process. Ex: Medicine, Education, Finance etc.
- Once the domain is determined, determine the attributes of the objects to be clustered. (Ex: Title, Place, job etc zones).
- Determine the strength of the relationships between the attributes whose co-occurrence in objects suggest those objects should be in the same class.
- Apply some algorithm to determine the class(s) to which each item will be assigned.

Guidelines

- Clear Meaning: Each cluster should have a clear semantic meaning.
- Balanced Size: Clusters should be roughly equal in size.If one cluster contains 90% of the items, it's not helpful.
- For example, assume a thesaurus class called "computer" exists and it contains the objects (words/word phrases) "microprocessor," "286-processor," "386- processor" and "pentium." If the term "microprocessor" is found 85 per cent of the time and the other terms are used 5 per cent each, there is a strong possibility that using "microprocessor" as a synonym for "286- processor" will introduce too many errors. It may be better to place "microprocessor" into its own class.
- Whether an object can be assigned to multiple classes or just one must be decided at creation time.
- There are additional important decisions associated with the generation of thesauri that are not part of item clustering. They are

1) Word coordination approach: specifies if phrases as well as individual terms are to be clustered

2) Word relationships: Aitchison and Gilchrist specified three types of relationships: equivalence, hierarchical and nonhierarchical. Equivalence relationships are the most

common and represent synonyms. Hierarchical relationships where the class name is a general term and the entries are specific examples of the general term. The previous example of “computer” class name and “microprocessor,” “pentium,” etc Non-hierarchical relationships cover other types of relationships such as “object”-“attribute” that would contain “employee” and “job title.”

3) Homograph resolution: a homograph is a word that has multiple, completely different meanings. For example, the term “field” could mean a electronic field, a field of grass, etc.

4) Vocabulary constraints: this includes guidelines on the normalization and specificity of the vocabulary. Normalization may constrain the thesaurus to stems versus complete words.

Thesaurus Generation

- There are three basic methods for generation of a thesaurus; hand crafted, co- occurrence, and header-modifier based.

1. **Hand-Crafted Thesauri**	2. **Co-Occurrence-Based Thesauri**	3. **Header-Modifier (Linguistic) Based Thesauri**
Created manually by domain experts. Example: A thesaurus for medical terms. Works best in specific domains (e.g., legal, medical). General thesauri (like WordNet) are less helpful for search and query expansion due to multiple meanings (polysemy).	Automatically generated based on how often words appear together in a text. Reflects real usage in documents. Relies on statistical methods. Effective in capturing term similarity.	Based on linguistic parsing of text. Words are grouped based on grammatical roles they play. Assumption: Words in similar grammatical structures are similar in meaning. **Key Syntactic Structures Parsed**: Subject-Verb (e.g., "Dogs bark") Verb-Object (e.g., "Eat food") Adjective-Noun (e.g., "blue sky") Noun-Noun (e.g., "data analysis") **Co-occurrence + Mutual Information**: Each noun is associated with verbs, adjectives, and nouns it appears with. Mutual Information (typically with a log function) measures how strongly two words are associated. This score is used to compute similarity between words, helping in term classification.

- There are **two types** generation of a thesaurus
  - **Manual Clustering**
  - **Automatic Term Clustering**
    - **Complete Term Relation Method**
    - **Clustering Using Existing Clusters**
    - **One Pass Assignments**

Manual Clustering

- Steps
  1. Define the Domain:
     - Select the domain (e.g., data processing, medical, legal) the thesaurus is meant for.
     - Helps avoid homographs (same word, different meanings) and keeps terms domain-relevant
  2. Generate Word List Using Concordances:
     - Use tools like:
       - Existing thesauri
       - Concordances (alphabetical list of words with frequency and document references)
       - Domain dictionaries
  3. Select Useful Terms
     - Avoid unrelated or overly common words (like “computer” in a computer thesaurus).
     - Focus on **domain-relevant, meaningful terms**.
  4. Cluster the Selected Terms
     - Based on **semantic relationships** and **usage patterns**.
     - Done using **editor's judgment**—not automated.
     - Example: Group "memory", "chips", and "RAM" together if they co-occur in similar contexts
  5. Quality Assurance
     - Other editors review and refine the thesaurus.
     - Ensures **accuracy**, **consistency**, and **relevance**.

- **KWOC** (Key Word Out of Context)
  1. KWOC **does not provide context**—we can’t tell what kind of "chips" it is
- **KWIC** (Key Word In Context)
  1. Displays the term inside a sentence, showing the term with its surrounding words.
  2. Helps resolve ambiguities (e.g., memory chips vs wood chips).
- **KWAC (Key Word And Context)**
  1. Displays **keyword followed by full sentence context**.
  2. Easier for manual scanning of how a term is used in actual sentences.

The Figure shows the various displays for "**computer design contains memory chips**".

– The phrase is assumed to be from doc4; the other frequency and document ids for KWOC were created for this example.

– In the Figure 6.1 the character "/" is used in KWIC to indicate the end of the phrase.
– The KWIC and KWAC are useful in determining the meaning of homographs.

– The term "chips" could be wood chips or memory chips.

– In both the KWIC and KWAC displays, the editor of the thesaurus can read the
sentence fragment associated with the term and determine its meaning.

KWOC

TERM	FREQ	ITEM Ids
chips	2	doc2, doc4
computer	3	doc1, doc4, doc10
design	1	doc4
memory	3	doc3, doc4, doc8, doc12

KWIC

chips/	computer design contains memory
computer	design contains memory chips/
design	contains memory chips/ computer
memory	chips/ computer design contains

KWAC

chips	computer design contains memory chips
computer	computer design contains memory chips
design	computer design contains memory chips
memory	computer design contains memory chips

Figure 6.1 Example of KWOC, KWIC and KWAC

- Once the terms are selected, they are clustered based upon the word relationship guidelines and the interpretation of the strength of the relationship.
- The resultant thesaurus undergoes many quality assurance reviews by additional editors using some of the guidelines already suggested before it is finalized

Automatic Term Clustering

- The more frequently two terms co-occur in the same items, the more likely they are about the same concept.
- They differ by the completeness with which terms are correlated.
- The more complete the correlation, the higher the time and computational overhead to create the clusters.
- The most complete process computes the strength of the relationships between all combinations of the "n" unique words with an overhead of other techniques start with an arbitrary set of clusters and iterate on the assignment of terms to these clusters.
- The basis for automatic generation of a thesaurus
  - Is a set of items that represents the vocabulary to be included in the thesaurus.
  - Selection of this set of items is the first step of determining the domain for the thesaurus.
  - The processing tokens (words) in the set of items are the attributes to be used to create the clusters.
- The basis for automated method of clustering documents
  - Is based upon the polythetic clustering where each cluster is defined by a set of words and phrases.

- Inclusion of an item in a cluster is based upon the similarity of the item's words and phrases to those of other items in the cluster.

Two Techniques in Automatic Term Clustering

1. **Full Correlation Matrix Method**
   - This method computes all **pairwise correlations between terms**.
   - If there are n unique words, this results in a time and **computational overhead** of **$O(n^2)$** — meaning:
   - Time increases **quadratically** with the number of words.
   - It is very **expensive and slow when n is large**.
2. **Iterative Clustering Method**
   - Begins with an **initial set of clusters** (could be random).
   - Then, it **iteratively reassigns terms** to clusters based on similarity.
   - If a **large number of clusters** is created initially, it can be used to build higher-level clusters (a hierarchy or layered grouping).

Automatic Term Clustering

- **Complete Term Relation Method**
  - Similarity between every term pair is calculated for determining the clusters.
  - The vector model is represented by a matrix where the rows are individual items and the columns are the unique words (processing tokens) in the items.
  - The values in the matrix represent how strongly that particular word represents concepts in the item.
  - Figure 6.2 provides an example of a database with 5 items and 8 terms. To determine the relationship between terms, a similarity measure is required. The measure calculates the similarity between two terms.

- where “k” is summed across the set of all items. In effect the formula takes the two columns of the two terms being analyzed, multiplying and accumulating the values in each row.
- The results can be paced in a resultant “m” by “m” matrix, called a Term-Term Matrix (Salton-83), where “m” is the number of columns (terms) in the original matrix.
- This simple formula is reflexive so that the matrix that is generated is symmetric. Other similarity formulas could produce a non- symmetric matrix.

	Term1	Term2	Term3	Term4	Term5	Term6	Term7	Term8
Item 1	0	4	0	0	0	2	1	3
Item 2	3	1	4	3	1	2	0	1
Item 3	3	0	0	0	3	0	3	0
Item 4	0	1	0	3	0	0	2	0
Item 5	2	2	2	3	1	4	0	2

Figure 6.2 Vector Example

in understanding the methodology so the following simple measure is used:

$$SIM(Term_i, Term_j) = \Sigma (Term_{k,i}) (Term_{k,j})$$

- Using the data in Figure 6.2, the Term-Term matrix produced is shown in Figure 6.3.
- There are no values on the diagonal since that represents the auto correlation of a word to itself.
- The next step is to select a threshold that determines if two terms are considered similar enough to each other to be in the same class.
- In this example, the threshold value of 10 is used.
- Thus, two terms are considered similar if the similarity value between them is 10 or greater.
- A new binary matrix called the Term Relationship matrix (Figure 6.4) that defines which terms are similar.
- A one in the matrix indicates that the terms specified by the column and the row are similar enough to be in the same class.
- Term 7 demonstrates that a term may exist on its own with no other similar terms identified.
- In any of the clustering processes described below this term will always migrate to a class by itself.
- The final step in creating clusters is to determine when two objects (words) are in the same cluster.

	Term1	Term2	Term3	Term4	Term5	Term6	Term7	Term8
Term 1		7	16	15	14	14	9	7
Term 2	7		8	12	3	18	6	17
Term 3	16	8		18	6	16	0	8
Term 4	15	12	18		6	18	6	9
Term 5	14	3	6	6		6	9	3
Term 6	14	18	16	18	6		2	16
Term 7	9	6	0	6	9	2		3
Term 8	7	17	8	9	3	16	3	

Figure 6.3 Term-Term Matrix

	Term1	Term2	Term3	Term4	Term5	Term6	Term7	Term8
Term 1		0	1	1	1	1	0	0
Term 2	0		0	1	0	1	0	1
Term 3	1	0		1	0	1	0	0
Term 4	1	1	1		0	1	0	0
Term 5	1	0	0	0		0	0	0
Term 6	1	1	1	1	0		0	1
Term 7	0	0	0	0	0	0		0
Term 8	0	1	0	0	0	1	0	

Figure 6.4 Term Relationship Matrix

The following algorithms are the most common: **cliques, single link, stars and connected components.**

Cliques:

Cliques require all terms in a cluster to be within the threshold of all other terms.

0. Let i = 1
1. Select $term_i$ and place it in a new class
2. Start with $term_k$ where r = k = i + 1
3. Validate if $term_k$ is within the threshold of all terms within the current class
4. If not, let k = k + 1
5. If k > m (number of words)

Applying the algorithm to Figure 6.4, the following classes are created:

Class 1 (Term 1, Term 3, Term 4, Term 6)
Class 2 (Term 1, Term 5)
Class 3 (Term 2, Term 4, Term 6)
Class 4 (Term 2, Term 6, Term 8)
Class 5 (Term 7)

then r = r + 1
if r = m then go to 6 else
k = r
create a new class with $term_i$ in it
go to 3
else go to 3

6. If current class only has $term_i$ in it and there are other classes with $term_i$ in them
then delete current class
else i = i + 1
7. If i = m + 1 then go to 8
else go to 1
8. Eliminate any classes that duplicate or are subsets of other classes.

Single Link:

- Any term that is similar to any term in the cluster can be added to the cluster.
- It is impossible for a term to be in two different clusters.
- Applying the algorithm for creating clusters using single link to the Term Relationship Matrix, Figure 6.4, the following classes are created:

1. Select a term that is not in a class and place it in a new class
2. Place in that class all other terms that are related to it
3. For each term entered into the class, perform step 2
4. When no new terms can be identified in step 2, go to step 1.

Class 1 (Term 1, Term 3, Term 4, Term 5, Term 6, Term 2, Term 8)
Class 2 (Term 7)

Star:

- Select a term and then places in the class all terms that are related to that term.
- Terms not yet in classes are selected as new seeds until all terms are assigned to a class.
- There are many different classes that can be created using the Star technique.
- If we always choose as the starting point for a class the lowest numbered term not already in a class.

Class 1 (Term 1, Term 3, Term 4, Term 5, Term 6)
Class 2 (Term 2, Term 4, Term 8, Term6)
Class 3 (Term 7)

String:

- Starts with a term and includes in the class one additional term that is similar

Class 1 (Term 1, Term 3, Term 4, Term 5, Term 6)
Class 2 (Term 2, Term 4, Term 8, Term6)
Class 3 (Term 7)

to the term selected and not already in a class.

- The new term is then used as the new node and the process is repeated until no new terms can be added because the term being analyzed does not have another term related to it or the terms related to it are already in the class.
- A new class is started with any terms not currently in any existing class.
- Using the guidelines to select the lowest number term similar to the current term and not to select any term already in an existing class produces the following classes.

Network Diagram of term similarities

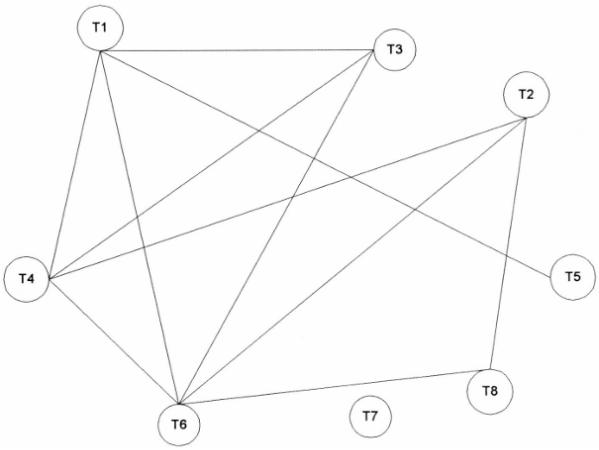


Figure 6.5 Network Diagram of Term Similarities

Comparison:

Aspect	Clique	Single Link
Class Relationship Strength	Strongest relationship between all terms in a class	Weakest relationship; a term connects to at least one in the class
Concept Representation	More likely to describe a **specific concept**	Covers **diverse/unrelated concepts**
Number of Classes Produced	Produces **more classes**	Produces the **fewest classes**
Similarity Requirement	All terms must be **mutually similar**	Terms can be grouped even with **zero similarity**
Use Case (Matrix Density)	Best for **dense** term relationship matrices	Best for **sparse** term relationship matrices
Precision vs Recall	**High precision** – ideal for query term expansion	**High recall** – may retrieve more non-relevant items
Computational Overhead	Higher (not specified, but more complex due to full linkage requirement)	**Lower overhead** – only **$O(n^2)$** comparisons
Use in Thesaurus Construction	Suitable when **concept clarity** is critical	Suitable when **broad coverage** is desired

Automatic Term Clustering

- **Clustering Using Existing Clusters**
  - Start with a set of existing clusters.
  - The initial assignment of terms to the clusters is arbitrary and revised by revalidating every term assignment to a cluster.
  - To minimum calculations, centroids are calculated for each cluster.
  - Centroid: the average of all of the vectors in a cluster.
  - The similarity between all existing terms and the centroids of the clusters can be calculated.
  - The term is reallocated to the cluster that has the highest similarity.
  - The process stops when minimal movement between clusters is detected.

Step 1: Initial Cluster Centroids

- Black squares = initial centroids.
- Triangles = terms.
- Circles = initial groupings of terms into 3 clusters (Classes 1, 2, 3).
- At this stage, the centroids are arbitrarily placed and do not accurately represent the final clusters yet.

Class 1 = (Term 1, Term 2)

Class 2 = (Term 3, Term 4)

Class 3 = (Term 5, Term 6)

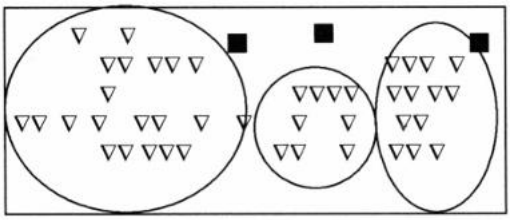


Figure 6.6b. Initial Centroids for Clusters

Step 2: After Reassignment

- After the first iteration of recalculating centroids and reassigning terms, we see:
- The black squares have moved closer to actual clusters.
- Terms are more appropriately grouped.
- Still not perfect, but closer to ideal.

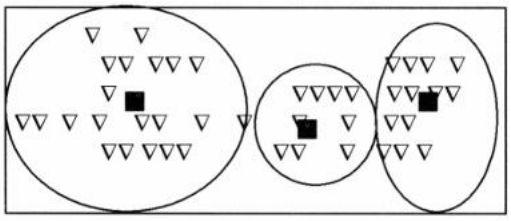


Figure 6.6a Centroids after Reassigning Terms

Step 3: Centroid Calculations

- Class 1 = (Term 1 and Term 2),
- Class 2 = (Term3 and Term 4) and Class 3 = (Term5 and Term 6).

Class 1 = (0 +4)/2, (3 + 1)/2, (3 + 0)/2, (0 +1)/2, (2 + 2)/2
= 4/2, 4/2, 3/2, 1/2, 4/2

Class 2 = 0/2, 7/2, 0/2, 3/2, 5/2

Class 3 = 2/2 3/2, 3/2, 0/2, 5/2

Step 4: First Iteration of Term Assignment

- Apply the simple similarity measure between each of the 8 terms and 3 centroids.
- One technique for breaking ties is to look at the similarity weights of the other terms in the class and assign it to the class that has the most similar weights.

	Term1	Term2	Term3	Term4	Term5	Term6	Term7	Term8
Class 1	29/2	29/2	24/2	27/2	17/2	32/2	15/2	24/2
Class 2	31/2	20/2	38/2	45/2	12/2	34/2	6/2	17/2
Class 3	28/2	21/2	22/2	24/2	17/2	30/2	11/2	19/2
Assign	Class2	Class1	Class2	Class2	Class3	Class2	Class1	Class1

Figure 6.7 Iterated Class Assignments

Step 5: New Centroids

- After new assignments, **centroids are recalculated** again. These are based on the **new groups** of terms
- **Only Term 7** moved from Class 1 to Class 3, which makes sense because:
- Its similarity to Class 1 was weak.
- It's closer to Class 3's centroid now.

Class 1 = 8/3, 2/3, 3/3, 3/3, 4/3
Class 2 = 2/4, 12/4, 3/4, 3/4, 11/4
Class 3 = 0/1, 1/1, 3/1, 0/1, 1/1

	Term1	Term2	Term3	Term4	Term5	Term6	Term7	Term8
Class 1	23/3	45/3	16/3	27/3	15/3	36/3	23/3	34/3
Class 2	67/4	45/4	70/4	78/4	33/4	72/4	17/4	40/4
Class 3	12/1	3/1	6/1	6/1	11/1	6/1	9/1	3/1
Assign	Class2	Class1	Class2	Class2	Class3	Class2	Class3	Class1

Figure 6.8 New Centroids and Cluster Assignments

Clustering Using Existing Clusters:

- Computation overhead: O(n).
- The number of classes is defined at the start of the process and cannot grow.
- It is possible to be fewer classes at the end of the process.
- Since all terms must be assigned to a class, it forces terms to be allocated to classes, even if their similarity to the class is very weak compared to other terms assigned.

Automatic Term Clustering

- **One Pass Assignments**
  - Minimum overhead: only one pass of all of the terms is used to assign terms to classes.
  - **Algorithm**
    - The first term is assigned to the first class.
    - Each additional term is compared to the centroids of the existing classes.
    - A threshold is chosen. If the item is greater than the threshold, it is assigned to the class with the highest similarity.
    - A new centroid has to be calculated for the modified class.
    - If the similarity to all of the existing centroids is less than the threshold, the term is the first item in a new class.
    - This process continues until all items are assigned to classes.

- Example with threshold of 10:

Class generated

- Class 1 = Term 1, Term 3, Term 4
- Class2 = Term 2, Term 6, Term 8
- Class 3 = Term 5
- Class 4 = Term 7

Centroids values used

- Class 1 (Term 1, Term 3) = 0, 7/2, 3/2, 0, 4/2
- Class 1 (Term 1, Term 3, Term 4) = 0, 10/3, 3/3, 3/3, 7/3
- Class 2 (Term 2, Term 6) = 6/2, 3/2, 0/2, 1/2, 6/2

- Minimum computation on the order of O(n).
- Does not produce optimum clustered classes.
- Different classes can be produced if the order in which the items are analyzed changes.

Item Clustering

- Think about manual item clustering, which is inherent in any library or filing system – one item one category.
- Automatic clustering – one primary category and several "secondary" categories
- Similarity between documents is based on two items that have terms in common.
  - The similarity function is performed between rows of the item matrix

Step 1: Calculating Similarity

- Formula **measures similarity** between two items by computing the **dot product** of their term vectors.
- $Term_{i,k}$: Term weight or count of term k in Item i
- If two items share many common terms (or the same terms with high weight), their similarity is high.

$$SIM(Item_i, Item_j) = \Sigma\ (Term_{i,k})\ (Term_{j,k})$$

Step 2: Item/Item Similarity Matrix

- Higher number → greater similarity.
- For example: Item 2 and Item 5 have similarity 36 (strong connection).

	Item 1	Item 2	Item 3	Item 4	Item 5
Item 1		11	3	6	22
Item 2	11		12	10	36
Item 3	3	12		6	9
Item 4	6	10	6		11
Item 5	22	36	9	11	

Figure 6.9 Item/Item Matrix

Step 3: Item Relationship Matrix

- We apply a threshold (e.g., 10):
- If similarity ≥ 10 → assign 1
- Else → assign 0
- This results in a binary matrix (Figure 6.10) representing item relationships.

	Item1	Item2	Item3	Item4	Item5
Item1		1	0	0	1
Item2	1		1	1	1
Item3	0	1		0	0
Item4	0	1	0		1
Item5	1	1	0	1	

Figure 6.10 Item Relationship Matrix

Step 4: Clustering Methods

1. **Clique Algorithm:**

- Finds **fully connected subgroups**.

  Class 1 = Item 1, Item 2, Item 5
  Class 2 = Item 2, Item 3
  Class 3 = Item 2, Item 4, Item 5
- Note: Items can belong to multiple cliques.

2. **Single Link Algorithm:**

- Builds a single cluster by linking items that are indirectly connected.

  Class 1 = Item 1, Item 2, Item 5, Item 3, Item 4

  Because:

  Item 1 → Item 2 → Item 3 & 5 → Item 4
- All linked through at least one neighbor.

3. **Star Algorithm:**

- Starts with the **lowest unassigned item** as the center.

  Class 1 - Item 1, Item 2, Item 5
  Class 2 - Item 3, Item 2
  Class 3 - Item4, Item2, Item5
- Each star has a center with its directly connected items.

4. **String Algorithm:**

- Creates **chains of items** until all are assigned.

  Class 1 - Item 1, Item 2, Item 3
  Class 2 - Item 4, Item 5
- Grows clusters by connecting the next available unassigned item.

Step 5: Clustering with Centroids

- Instead of using relationships, now use **term vectors** to create **centroids** (average vectors of each class).
- **Initial Assignment:**
  Class 1: Items 1 & 3
  Class 2: Items 2 & 4
- **Compute Centroids (Averages of Item Vectors)**
- Example:
  Class 1 Centroid = (Item 1 + Item 3) / 2
  Class 2 Centroid = (Item 2 + Item 4) / 2
- Then compare each item's vector to both centroids, and **reassign** it based on higher similarity.

Class 1 = 3/2, 4/2, 0/2, 0/2, 3/2, 2/2, 4/2, 3/2
Class 2 = 3/2, 2/2, 4/2, 6/2, 1/2, 2/2, 2/2, 1/2

	Class1	Class 2	Assign
Item 1	33/2	17/2	Class 1
Item 2	23/2	51/2	Class 2
Item 3	30/2	18/2	Class 2
Item 4	8/2	24/2	Class 2
Item 5	31/2	47/2	Class 2

Figure 6.11 Item Clustering with Initial Clusters

Hierarchy of Clusters

- Hierarchical Clustering builds a **tree-like structure (called a dendrogram)** of clusters based on the similarity or distance between data points (documents or terms).
- Hierarchical agglomerative clustering (HAC) – start with **un-clustered items and perform pair-wise similarity measures to determine the clusters.**
- Hierarchical divisive clustering – start with a cluster and breaking it down into smaller clusters
- There are two main approaches:
- **Agglomerative (Bottom-up):** Start with each item in its own cluster and successively merge the most similar pairs.
- **Divisive (Top-down):** Start with one big cluster and divide it into smaller ones.

Objectives of Creating a Hierarchy of Clusters :

- **Reduce the overhead of search**
  - Perform top-down searches of the centroids of the clusters in the hierarchy and trim those branches that are not relevant.
- **Provide for visual representation of the information space**
  - Visual cues on the size of clusters (size of ellipse) and strengths of the linkage between clusters (dashed line, sold line…).
- **Expand the retrieval of relevant items**
  - A user, once having identified an item of interest, can request to see other items in the cluster.
  - The user can increase the specificity of items by going to children clusters or by increasing the generality of items being reviewed by going to a parent clusters.

Dendogram for Visualizing Hierarchical Clusters

- **Hierarchical Agglomerative Clustering Method (HACM) :**
  - First the N * N item relationship matrix is formed.
  - Each item is placed into its own clusters.
  - The following two steps are repeated until only one cluster exists
    - The two clusters that have the highest similarity are found
    - These two clusters are combined, and the similarity between the newly formed cluster and the remaining clusters recomputed
  - As the larger cluster is found, the clusters that merged together are tracked and form a hierarchy
- HACM Example :
  - Assume document A, B, C, D, and E exist and a document-document similarity matrix exists .
  - {{A} {B} {C} {D} {E}} ➔ {{A, B} {C} {D} {E}} ➔ …➔ {A, B, C, D, E}

- **Similarity Measure between Clusters**:
  - Single link clustering
    - The similarity between two clusters (inter-cluster similarity) is computed as the maximum similarity between any two documents in the two clusters, each initially from a separate.
  - Complete linkage
    - Inter-cluster similarity is computed as the minimum of the similarity between any documents in the two clusters such that one document is from each cluster.
  - Group average
    - As a node is considered for a cluster, its average similarity to all nodes in the cluster is computed. It is placed in the cluster as long as its average similarity is higher than its average similarity for any other cluster
- **Cluster Similarity: Lance-Williams Formula**

  The **Lance-Williams formula** is a generic formula for updating dissimilarities during clustering:

  $$D(C_{ij}, C_k) = \alpha_i D(C_i, C_k) + \alpha_j D(C_j, C_k) + \beta D(C_i, C_j) + \gamma |D(C_i, C_k) - D(C_j, C_k)|$$

  - Used to calculate the distance between a new merged cluster C_{ij} and other clusters.
  - By selecting values of $\alpha_i, \alpha_j, \beta, \gamma$, you can simulate different clustering methods like single linkage, complete linkage, average linkage, etc.
  -
- **Ward's method**
  - Clusters are joined so that their merger minimizes the increase in the sum of the distances from each individual document to the centroid of the cluster containing it.
  - If cluster A merged with either cluster B or cluster C, the centroids for the potential cluster AB and AC are computed as well as the maximum distance of any document to the centroid. The cluster with the lowest maximum is used.

$$I_{ij} = \frac{(m_i m_j)}{(m_i + m_j)} d_{ij}^2$$

$$d_{ij}^2 = \sum_{k=1}^{n} (x_{i,k} - x_{j,k})^2$$

Where:

- m_i, m_j: number of items in clusters.
- d_{ij}^2: squared Euclidean distance between centroids.

This method is often preferred for clustering documents because it tends to create compact clusters.

- **Analysis of HACM**:
  - Ward's method typically took the longest to implement.
  - Single link and complete linkage are somewhat similar in run time.
  - Clusters found in the single link clustering tend to be fair broad in nature and provide lower effectiveness.
  - Choosing the best cluster as the source of relevant documents results in very close effectiveness results for complete link, Ward's and group average clustering.
  - A consistent drop in effectiveness for single link clustering is noted.

- **Monolithic vs Polythetic Clusters**
  - Monolithic: Clusters are formed based on a single attribute; easier to interpret (e.g., Yahoo directories).
  - Polythetic: Based on multiple features like terms and concepts; harder to interpret but more flexible.
- **Concept Hierarchy with Term Subsumption (Sanderson & Croft)**
  - If 80% of the documents containing term Y also contain term X → X is a more general concept than Y.
  - This builds a concept hierarchy where:
  - Parent = more general term
  - Child = more specific term
  - Represented as a directed acyclic graph (DAG)

$$P(X|Y) \geq 0.8, \quad P(Y|X) < 1$$

- **Semantic Hierarchies and Thesauri**
  - Manual thesauri use semantic relationships (e.g., is-a, part-of).
  - Automatic clustering can build item hierarchies effectively.
  - However, for term hierarchies, automation often introduces errors, so human-guided construction is more reliable.

Component	Explanation
Dendrogram	Tree diagram showing how documents/items are hierarchically clustered
HACM	Hierarchical Agglomerative Clustering Methods used to group similar documents
Lance-Williams Formula	General formula to update cluster distances
Ward's Method	Minimizes variance within clusters
Subsumption Rule	Defines parent-child relationship in term hierarchies
Centroids	Mean vector of items in a cluster; used to cluster at higher levels
Scatter/Gather	IR technique using repeated clustering
Monolithic vs Polythetic	Monolithic: simple, one-topic clusters; Polythetic: multi-attribute, harder to interpret